arXiv:2609.35370v1 [cs.AI] 28 Sep 2026

INTERNATIONAL TRANSACTIONS IN OPERATIONAL RESEARCH

Intl. Trans. in Op. Res. XX (20XX) 1–30
DOI: xx.xxxx/itor.xxxxx

A decision-support system applied to Law: Reasoning and explainability of the decision

Jeremy BOUCHE-PILLON[a,*], Pascale ZARATE[a,b], Yannick CHEVALIER[a,c] and Nathalie AUSSENAC-GILLES[a]

[a] *Université de Toulouse - CNRS - IRIT, France*
[b] *Université Toulouse Capitole, France*
[c] *Université Toulouse 3 Paul Sabatier, France*
E-mail: Jeremy.Bouche-Pillon@irit.fr [First Author]; Pascale.Zarate@ut-capitole.fr [S. Author]; Yannick.Chevalier@irit.fr [T. Author]; Nathalie.Aussenac-Gilles@irit.fr [Fourth Author]

Received DD MMMM YYYY; received in revised form DD MMMM YYYY; accepted DD MMMM YYYY

Abstract

The emergence of the digital transition brought an increasing need to control the processing of digital information, including in Law Enforcement Agencies (LEAs). At the EU level, in recent years, many regulations have emerged to control data processing and exchange. Texts other than the GDPR, such as the ”Law Enforcement Directive (LED)”, appeared to regulate specifically how Law Enforcement Agencies (LEAs) could process data. A formal representation of these regulations can be part of decision systems that support LEAs in processing data in compliance with the regulations. Although many new formalisms have emerged to represent legal norms and rules, few are provided with a reasoning mechanism. Furthermore, systems used in decision-making processes in critical contexts such as medical diagnoses or legal decisions cannot be fully automated, and the explainability of their results is essential to ensure user confidence in decisions. This explainability aspect, while crucial, is lacking in most modern approaches that rely on machine learning. This paper describes a framework to operate formal rules from regulations, by focusing on explainability of the decision. After describing the general architecture of the proposed decision support framework, the paper showcases how symbolic AI and the SPARQL query language can support legal reasoning. It then describes an algorithm to generate a justification for the reasoning results, and outlines the procedure to be followed when the reasoning does not lead to a satisfactory conclusion. We notably focus on a method based on decision trees to determine what additional information to request from the user.

Keywords: decision-support; rule-based; decision trees; law compliance; explainability

[*] Author to whom all correspondence should be addressed (e-mail: Jeremy.Bouche-Pillon@irit.fr).

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

1. Introduction

Digital technologies present enormous growth potential. They promise to provide better and more seamless service and market access to citizens and economic agents by replacing existing services with digital ones that implement best-of-class solutions, are interoperable, and support agents in their tasks. To be accepted, this digital transition[1] requires comprehensive governance to prevent the misuse of the introduced technology. Governance is often domain-specific and is defined through laws establishing guidelines and barriers on what can and cannot be done in finance, e-Health, administration, etc. For example, while the GDPR[2] applies to all digital processing of EU citizens' personal data, the Digital Service Act (DSA)[3] applies more specifically to personal data processed in marketplaces and social networks.

These regulations force agents willing to transform their activities to prove that their processes comply with laws whose interpretation may vary. A solution is to introduce a Decision Support System (DSS) in which a qualified human agent is responsible for the description of the task at hand, and the system suggests a proper course of action to ensure compliance. To ensure that the human agent understands the legal implications, this advice should be supported with a legal reasoning justification based on the existing regulations. Such a DSS requires the formalization of regulations and reasoning on this formalization, as well as an appropriate communication with the human agent.

Building such a DSS is challenging, as individual components themselves need to solve complex problems, and the provided solutions need to be compatible across the system. For example, many formalisms have been proposed to represent legal norms and rules, the prime examples being LegalRuleML (Palmirani et al., 2011) and LKIF (Gordon, 2008). However, this formalization needs to be operational, and to that end needs to be accompanied by a working mechanism for reasoning on the rules. For instance, LegalRuleML did not originally provide mechanisms to reason over the rules and supplementary works had to be carried out to allow it (Lam and Hashmi, 2018). The Carneades argumentation system (Gordon, 2008), designed to work with LKIF rules, could not be used successfully when tested and doesn't appear to be maintained, which shows the need for a standard-based solution.

Another major issue, once the rules and the situation are described in a suitable logic, is the inability of pure logic reasoning mechanisms to explain the decision, only stating whether a decision is a logical consequence of its premises. That is, the pure reasoning should be complemented with a module that reflects upon the process that led to the result to provide the user with meaningful information.

Finally, the growing complexity of the relations between stored pieces of information led to represent them using a standard format from Semantic Web technologies. Instead of relying on *ad hoc* relations stored in relational database tables, this framework promotes the use of well-defined entities and binary relations from standardized *ontologies* to store information in *Knowledge Graphs* (KG) as defined in Lazarska and Siedlecka-Lamch (2019); Medhi and Baruah (2017), and provides the ability to query these graphs using standard query languages that are broadly supported with tools.

Based on all these observations, we propose to support decision making when one has to check the conformance of a situation to a set of rules describing one or more regulations. In this paper, we will notably focus on establishing a method for generating explanations of the decision suggested. A state of

[1]Digital transition - Reforms and Investments - European Union: https://reforms-investments.ec.europa.eu/technical-support-instrument-0/digital-transition_en

[2]Regulation - 2016/679 - EN - gdpr - EUR-Lex - European Union: https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng

[3]Regulation - 2022/2065 - EN - DSA - EUR-Lex: https://eur-lex.europa.eu/eli/reg/2022/2065/oj/eng

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

the art on decision support systems presented in a previous paper (Bouché-Pillon, 2025) led us to consider a symbolic approach, as opposed to learning-based approaches, given the critical nature of legal compliance checking. This approach also requires the development of methods for formally representing regulatory knowledge, legal rules and notions. The aim of this work is to answer the following research questions: (RQ1) How can symbolic AI be used to reason over formalized legal texts to verify compliance and provide justifications ? (RQ2) How to integrate semantic web standards in legal reasoning through the formalization of regulations? (RQ3) How to ensure that the results from legal reasoning provide a justified suggestion to the user? (RQ4) How to identify the most relevant information to ask a user when faced with an undecided scenario ?

We propose to answer these questions by designing a framework that takes as input a situation description by a human agent using a specific ontology, and that supports reasoning on this situation thanks to formal rules represented using the Semantic Web's standard SPARQL. This part raises RQ1 and RQ2. If a decision can be reached, relevant legal sources are provided to support it. In the event of a failure, which occurs either in case of an inconsistency in the applicable rules or if the situation is insufficiently described, this reasoning module is complemented by an explainability module. This second module can analyze the cause of the failure and then ask the user for any missing information, or report any inconsistency met. Setting up this module requires to answer RQ3 and RQ4.

To illustrate the design and use of this framework, we selected a use case about checking the conformance of data sharing and processing in LEAs to several European regulations. The central piece of legislation is the "Law Enforcement Directive" (LED)[4], which aims at regulating the processing of data related to investigations by Law Enforcement Agencies (LEAs) while respecting the right to privacy of EU citizens. This restricted scope makes this case tractable in terms of rule and situation complexity. In spite of this, it is rich enough to capture important concepts such as privacy, agents, processing, and the laws that can be stated over these. Previous works have already covered the creation of the ontology (Bouché-Pillon et al., 2024) and of the formal rules (Bouche-Pillon et al., 2024) used in our studied use case. This paper focuses on developing an algorithm for generating an explanation to the rule-based reasoning results.

Outline. We first present the overall architecture of the framework and the relations between its components in Section 2. Section 3 introduces the formalism used to represent rules taken from regulations. It is continued in Section 4 in which the different types of results that can be obtained through reasoning over these rules are presented, including the case of an undecided result. This case is handled by the explainability module described in Section 5. In particular, since the undecided cases are caused by a lack of information in the description of the situation, this section presents the procedures employed to identify the supplementary information that should be asked of the user. This paper concludes in Section 6 with a summary on the presented works and a presentation of research perspectives.

[4]2016/680 - EN - Law Enforcement Directive; LED - EUR-Lex: https://eur-lex.europa.eu/eli/dir/2016/680/oj/eng

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

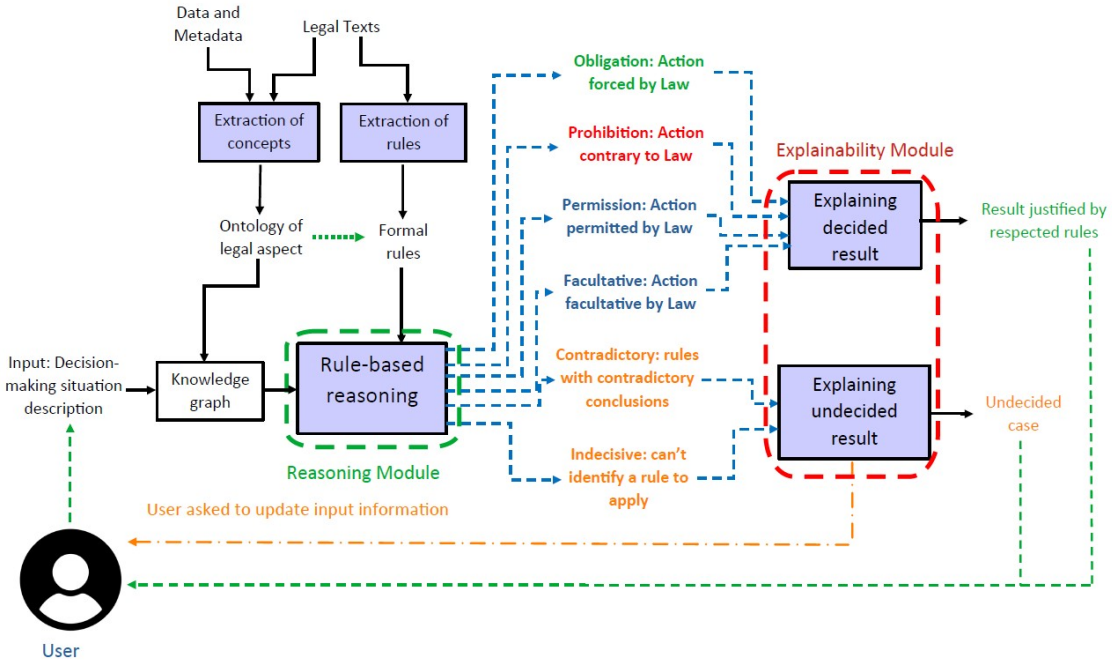


Fig. 1: Framework principle

2. Decision-Support Framework

Given the lack of explainable reasoning frameworks over formal legal rules, we aim to provide a DSS that interacts with the user to explain its decisions. To that end, we provide an interactive DSS by following the architecture described in (Marakas, 2003) to obtain both the operability of formal rules and the explainability of reasoning results.

2.1. General architecture

The DSS has been fully described in a previous paper (Bouche-Pillon et al., 2024). Its main components are depicted in Fig. 1.

- A first module, framed in green in Fig. 1, implements a rule-based approach to ensure the respect of Law and regulations. One of the major drawbacks of rule-based systems is their inability to provide a response in cases where none of the rules in the system rule base is triggered. There is also a risk of triggering several contradictory rules. Based on this observation, it is already possible to envisage 2 different types of results for this first module: (i) Cases where the rules manage to reach a *decided* conclusion, for example by stating that, *"given the situation described in the input, an action is permitted"*. (ii) In the other cases, the rules do not manage to reach a satisfactory conclusion, and

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

end up in an *undecided* state. The handling of the result will differ depending on which of these two situations the first module ends up in.

- A second module, framed in red in Fig. 1, is in charge of justifying the output of the reasoning module. While *decided* results are easily justifiable, *undecided* results do not provide a satisfactory answer. Therefore, it is advisable to improve the result before returning it is to the users, by asking them to adjust the input information so that the output could be *decided*.

The operation of these modules is based on several other components that are also illustrated in Fig. 1 and described in the remainder of this section.

2.2. Legal sources

The rule-based reasoning module relies on several components that contain information taken from legal sources such as laws and regulations. In most regulations, there are two types of information: (i) Definitions that clearly establish the terms and concepts involved, as well as the limitations of the regulations. These elements are integrated in the ontology and knowledge base, which will enhance the system's inference capabilities during the reasoning phase; (ii) The normative articles that use the terms and concepts defined earlier and establish legal rules. These rules are extracted and formalized to then serve as the basis of the reasoning module.

2.3. Ontology for the reasoning module

Ontologies are a first-order logic vocabulary of entities and relations aiming at capturing the concepts in a given domain with a set of rules, and usually rely on the concepts described in existing ontologies. For example, a hierarchy of types of agents can be defined with the names of the types, a relation giving a type to agents, and rules capturing that agents in a subclass are also agents in the superclass. Their standardised description in RDF or OWL is the basis of the Semantic Web[5]. Our DSS integrates a dedicated ontology to represent legal rules and situations in which a legal decision has to be computed. This ontology covers the representation of two types of legal knowledge:

- The knowledge related to the legal rules and their metadata, like for example the deontic modalities they contain, or the legal documents they come from. This part of the ontology is generic and can be used for any legal compliance checking scenario. This part of the ontology can be built by extracting concepts and metadata from legal documents like regulations.
- The knowledge related to the specific type of situations the DSS is applied. It must capture the characteristics of the data, people, and authorities involved in the decision-making process, as well as contextual information, such as whether the decision is made in the context of an emergency. This part of the ontology is unique for each possible use case of the framework, and must be built specifically for the use case prior to using the framework. Building this part of the ontology requires the analysis of the concepts related to the use case.

[5]Semantic Web: https://en.wikipedia.org/wiki/Semantic_Web

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

Accordingly, a situation is a *knowledge graph* (KG), *i.e.* a set of instances of the entities and relations in the ontology complemented with entities describing the situation. For example, a KG for our data-sharing use-case could have instances 'Alice `is a` *LEA agent*' and 'Bob `is a` *subject under investigation*' of the `'is a'` relation with the *'LEA agent'* and *'subject under investigation'* ontology-defined entities, and the situation-specific entities *'Alice'* and *'Bob'*. The rules of an ontology allows knowledge inference, that is, to reason over the most exhaustive description of a situation in a given use case. In the preceding example, instances stating that both Alice and Bob are agents are added to the set of instances. The encoded legal rules are also stored in a KG, and the rule-based reasoning module applies the rules stored in it to situations. The ontology employed was presented in a previous article (Bouché-Pillon et al., 2024), and the reasoning is described more precisely in the next three sections.

2.4. Input of the reasoning module

The input of the system consists of a description of the situation for which users want to check the conformance to legal rules. All the data involved in the decision-making process are filled in by users using a form and then encoded in a knowledge graph (KG), which is where the reasoning takes place. The KG used in our framework fulfills several roles: It stores the ontology to allow semantic reasoning on the input data, but also the metadata about the rules used in the reasoning module. These metadata cover rule characteristics such as their deontic type or the legal document they come from; The KG can also be used to store data that are not part of the user's query, but that are part of the user's organization database and are also available to reason with; Finally it is used to store the input information when a user makes a decision support request to the system. While the ontology and the metadata about the rules are stored in a *"global"* knowledge graph, two distinct approaches can be considered to store the input information:

1. The input information can also be stored in the global KG, alongside the ontology and the rules metadata. This approach is simple to implement, but it raises the issue of user query independence. Indeed, if several queries are made on the system, there is a risk that some of them define instances of the ontology with the same identifier while being fundamentally different. This problem could lead to flawed reasoning and, therefore, result in incorrect decision suggestions. One way to avoid this issue would be to implement a system that guarantees the separation of entity identifiers between different queries made to the system.
2. The other approach consists of storing each query in a specific named KG. This solution offers compartmentalization of the queries and prevents the reasoning process from being contaminated by external information unrelated to the query.

Adopting one of these approaches influences the way the rules need to be written. Within the scope of the proposed framework, we selected the approach with named graphs, where compartmentalization of user queries prevents flawed reasoning by design.

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

2.5. Formal rules of reasoning module

The formal rules are extracted and derived from regulations and other legal sources, and their metadata are expressed using the ontology. The rules themselves are written in SPARQL, a language that allows to query the KG. The metadata are then inserted into the KG used by the DSS. To encode legal reasoning and be able to provide useful feedback to the user, we differentiate several types of rules:

- Applicability rules that determine if legal articles apply to a situation, and allow for detecting when a situation is insufficiently described;
- Compliance rules that evaluate the compliance of a situation with legal articles;
- Exception rules that allow to define exception relationships between multiple rules, and thereby non-monotonic reasoning.

Each compliance rule is characterized by a deontic type (Permission, Facultative, Obligation or Prohibition) that is used in the reasoning process to evaluate the compliance of a situation with the regulations. It is important to note that the regulations from which the rules are extracted must, of course, be adapted to the scenario in which the framework is used.

2.6. Possible outcomes of the rule-based reasoning module

The first module, the **Rule-Based reasoning module**, develops a policy inspired by access control policies using formal rules extracted from regulations as well as concepts and relationships from the ontology. The two types of output, *decided* and *undecided* can be further divided using six output values.

The *decided* cases are divided in *Obligation* (green text in Fig. 1), *Prohibition* (red text in Fig. 1), *Permission* and *Facultative* (blue text in Fig. 1). These types of results are obtained when several rules are complied with, and all arrive at the same deontic conclusion.

The *undecided* cases (yellow text in Fig. 1) can be either *indecisive* or *contradictory*. The reasoning is indecisive when no single formal rule from the rule base has been triggered. It is contradictory if several compliance rules are applicable but give contradictory deontic answers. The handling of undecided cases calls for updating either the rule base or the input data, which occurs in the explainability module.

2.7. Explainability module

The second main module of the framework generates a justification for the conclusion of the rule-based reasoning module. It is returned to the user alongside the decision suggestion. Explainability is indeed an essential aspect in decision support systems, and particularly if these systems are used in critical contexts, such as for example establishing medical diagnoses or judging legal cases. This module takes the results from the rule-based reasoning and depending on the type of result, generates an appropriate justification.

In its current version, the proposed framework bases its explainability on the concept of *'Enacted Law'*, as opposed to *'Case Law'*. Rather than relying on the principle of precedence that would require an extensive database of previously determined cases, the justification of the decision comes from the

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

statutes such as laws and regulations. Relying on *'Enacted Law'* has two advantages in the scope of our study : First it provides a strict legal framework with which all institutions must comply, while *'Case Law'* can differ between institutions. Second, the language used for *'Enacted Law'* is very precise and defines the terms and their meanings for all parties affected by the law. By contrast, *'Case Law'* generally cannot be captured by a single authoritative and uncontroversial formulation[6].

Thus, in *decided* cases, the conclusion of the reasoning module is presented together with the legal sources of the rules that led to the conclusion as a justification.

The *undecided* cases however require special handling. In principle, *contradictory* scenarios should not occur. While such cases may arise from contradictions in the applicable law, they would most likely be caused by wrongly encoded rules or by missing exceptions in the rule base. When faced with a *contradictory* case, the system registers the conflict with the intention of forwarding it to an expert who will be able to determine how to resolve the conflict and update the rule base accordingly.

For *indecisive* cases, the system lacks information to reach a conclusion and will ask the user to update its input, while guiding him by indicating which information would lead to better chances of reaching a conclusion. Section 5 presents heuristics to identify information that, if added to the input, would allow the rule-based reasoning to reach a *decided* conclusion.

We will now take a look at the different types of rules that intervene in the rule-base reasoning process and the formalism we chose to represent them.

3. Using SPARQL for non monotonic reasoning

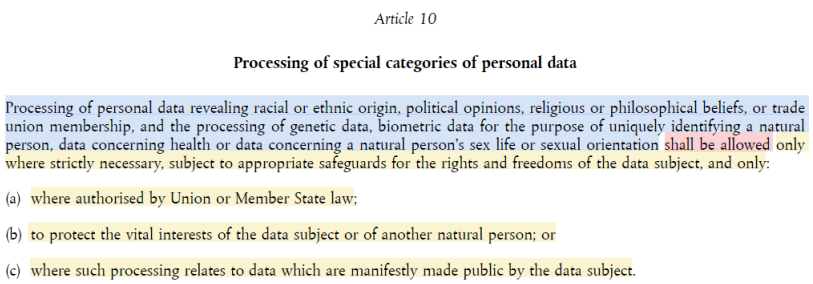
The proposed framework is designed to be used in various situations and contexts where it is desirable to determine whether an action is permitted, required, prohibited, or optional. Depending on the context, it is necessary to consider the relevant laws and regulations in order to extract not only the underlying concepts but also the rules that will govern the decision-support process. The rules obtained that way are "explicit" and can be automatically extracted. Other *implicit* rules, not explicitly present in regulations, are included to model "default cases" or exception rules defining when some rules should prevail over others. Both the rules and the input contexts are represented as a graph using the concepts and properties of the ontology (Bouché-Pillon et al., 2024).

It was decided to use SPARQL to express the formal rules, as it allows to reason over RDF and OWL data. Some works conducted on the semantics of this language showcased its expressivity and showed that, subject to a few minor restrictions, it is equivalent to first-order logic. Notably, it is possible to translate the body of SPARQL queries into First-Order Logic (FOL) formulas using Answer Set Programming (ASP) as an intermediate language (Polleres and Wallner, 2013; Lee and Palla, 2009).

The rules are applied using a SPARQL engine such as GraphDB[7] or Apache Jena Fuseki[8] to reason directly over knowledge graph content. However, for clarity reasons, in this paper, the formalization will be presented in a FOL form following a $Conditions \rightarrow Effect$ syntax. The SPARQL equivalents of the FOL expressions can be found on our Github repository[9] in the "rules" directory.

[6]IDENTIFYING APPLICABLE LAW - Law Explorer : https://lawexplores.com/identifying-applicable-law/
[7]GraphDB homepage: https://graphdb.ontotext.com
[8]Apache Jena Fuseki homepage: https://jena.apache.org/documentation/fuseki2/
[9]Github repository: https://github.com/JeremyBOUCHEPILLON/legalDataProcessing/

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies



Article 10

Processing of special categories of personal data

Processing of personal data revealing racial or ethnic origin, political opinions, religious or philosophical beliefs, or trade union membership, and the processing of genetic data, biometric data for the purpose of uniquely identifying a natural person, data concerning health or data concerning a natural person's sex life or sexual orientation shall be allowed only where strictly necessary, subject to appropriate safeguards for the rights and freedoms of the data subject, and only:

(a) where authorised by Union or Member State law;

(b) to protect the vital interests of the data subject or of another natural person; or

(c) where such processing relates to data which are manifestly made public by the data subject.

Fig. 2: Example of law article: article 10 from 2016/680/UE directive (LED)

3.1. Explicit rules from regulations

Explicit rules are those that can be directly extracted from the regulations in natural language. Although extraction is currently manual, rule extraction from regulations can be automated using LLMs and semantic analysis (Fawei, 2024; Ferraro et al., 2020; Recski et al., 2021), which we plan to do in future works. The formalization principle applied in this framework is adapted from Gandon et al.'s work (Gandon et al., 2017), where each rule extracted from the regulation does not indicate directly whether an action is permitted, prohibited or mandatory. Instead each rule is classified as either permission, obligation or prohibition and the reasoning aims at assessing the compliance of the input situation to each rule. Moreover, as explained in a previous paper (Bouche-Pillon et al., 2024), a legal rule is always encoded into two SPARQL queries, one indicating the applicability of the rule to the input situation, and the other indicating the respect of the rule by the situation. To illustrate this principle, the analysis of the article in Fig. 2 would decompose as follows:

(i) The deontic class of this article is "Permission", as indicated by the terms "shall be allowed". (ii) The object of the rule relates to the processing of "sensitive personal data". (iii) The conditions of this rule are "the strict necessity" of the processing, the "safeguard of rights and freedom of the data subject" and a disjunction of conditions: "allowed by Union or Member State" OR "protection of vital interests" OR "the data are public". From this analysis, the "permission" aspect of the rule is expressed by the axiom *isPermission(LED10)* and the rest can be formalized as the following 2 rules, in FOL:

- $Processing(action) \land InvolvesDataset(action, dataset) \land ContainsData(dataset, data) \land SensitivePersonalData(data) \rightarrow IsApplicable(LED10, situation)$

- $IsApplicable(LED10, situation) \land Action(action) \land InvolvesDataset(action, dataset) \land Necessary(action) \land SafeguardRights(action) \land (AuthorizedLaw(action) \lor ProtectsVitalInterest(action) \lor (\forall data, SensitivePersonalData(data) \implies PublicData(data))) \rightarrow HasCompliance(LED10, situation)$

Trying to write the second of these rules in SPARQL would give the query from listing 1

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

Listing 1: SPARQL query to express the compliance rule corresponding to the 10th article of the LED

```
INSERT { graph ?g { nru:PS1 nrv:hasCompliance ?g }}
WHERE {
        {graph ?g {
        ?action a :Processing .
        ?action :involvesData ?dataset .
        ?dataset :containsData ?data .
        ?data a :SensitivePersonalData .
        ?action :isNecessary "true"^^xsd:boolean .
        { ?action :isAuthorizedLaw "true"^^xsd:boolean }
        UNION
        { ?action :protectsVitalInterests "true"^^xsd:boolean }
        UNION
        { FILTER NOT EXISTS {
                ?data a :SensitivePersonalData .
                FILTER NOT EXISTS {
                ?data a :PublicData .
                }
        }}
}}}
```

The query can be obtained through the automatic translation from First-Order Logic (FOL) formulas, applying the works conducted by Polleres and Wallner (2013) and Pérez et al. (2009), using Answer-Set Programming (ASP) as an intermediate language. It can be noted that this translation introduces `FILTER NOT EXISTS` to express both the classical logical negation and the existential quantifier's negation. Since the universal quantifier does not exist in SPARQL, we go through the negation of the existential quantifier to express an equivalent formula: A formula of the form $\forall\, x, p(x)$ will be encoded in SPARQL as $\neg\, \exists\, x, \neg\, p(x)$.

Substitutions. Consider a partial mapping σ from the set of variables $\mathcal{X}$ to the set of terms. We extend it homomorphically to terms with $\sigma(t) = t$ when t is not a variable. Its *domain* is the minimal set of variables $D \subseteq \mathcal{X}$ such that $\sigma(x)$ is defined. It is idempotent if for all $x \in \mathcal{X}$ we have $\sigma(\sigma(x)) = \sigma(x)$. A *substitution* is a homomorphic extension to terms (and then to relations) of an idempotent partial mapping on variables of finite domain. Substitutions are usually denoted in the postfix notation, *i.e.* $x\sigma$ rather than $\sigma(x)$. We denote Σ_D a set of substitutions of domain D, and given $D' \subseteq D$, we denote $\pi_{D'}(\Sigma_D)$ the set of substitutions that are a restriction to D' of the substitutions in Σ_D. The *join* of two sets Σ_D and $\Sigma_{D'}$ of substitutions is denoted $\Sigma_D \bowtie \Sigma_{D'}$ and is the largest set of substitutions of domain $D \cup D'$ such that for all $\sigma \in \Sigma_D \bowtie \Sigma_{D'}$ we have $\pi_D(\sigma) \in \Sigma_D$ and $\pi_{D'}(\sigma) \in \Sigma_{D'}$. The union of two sets of substitutions of common domain D is simply denoted $\Sigma^1_D \cup \Sigma^2_D$. We note that there exists a unique set $\Sigma_\emptyset$ that contains the substitution of empty domain and that it is a neutral element for the join.

Knowledge Graphs. The information encoded in a FOL model can be encoded as a *Knowledge Graph* (KG) (within a larger domain and more predicates) with vertices representing entities and values, and arcs between 2 vertices labeled with a relation name. To bridge the gap with FOL models, a KG can also be seen as a unique relation between the head, the relation name, and the target of each arc.

Definition 1. (Knowledge Graph) A *Knowledge Graph* is a set of predicate instances $\texttt{PRED}(const, const, const)$.

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

SPARQLTree. SPARQL rules are of the form *body* $\rightarrow$ *effect* where the *body* designates a graph pattern formula. When applied on a knowledge graph K, the effect is applied once for each substitution σ such that $K \models \varphi\sigma$. To avoid the introduction of variables in K, we assume that the set of variables occurring in the effect is a subset of X. Additional quantifiers can be used in defining $\varphi^{X,W}$ through the `FILTER` construct. In the usual SPARQL syntax, a group $\left\{\varphi^{X,W}\texttt{FILTER}\psi^{Y\cup X,W'}\right\}$ has, as its set of solutions, the set of substitutions that are the restrictions on X of the substitutions of domain $X \cup Y$ that satisfy both $\varphi^{X,W}$ and $\psi^{Y\cup X,W'}$. This definition is not structural, which led us to the introduction of a structural version of SPARQL where the same construct is denoted $\texttt{FILTER}(\varphi^{X,W}, \psi^{X\cup Y,W'})$. This change allows for easily defining what $K \models \varphi$ means, and for the definition of mappings on SPARQL graph pattern formulas that will be developed *infra*. We have omitted some constructions, such as equality or typing constraints, to focus on those used in this article. The grammar for this SPARQLTree query body is, denoting *t* the terms that are either variables or (literal) constants and decorations thereof:

$$\begin{aligned} \textit{pred} &::= \texttt{PRED}(t,t,t) \\ \textit{body} &::= \texttt{AND}(\textit{body},\textit{body}) \mid \texttt{UNION}(\textit{body},\textit{body}) \mid \texttt{FILTER}(\textit{body},\textit{filter_constraint}) \\ \textit{filter_constraint} &::= \texttt{NOT}(\textit{filter_constraint}) \mid \texttt{EXISTS}(\textit{body}) \end{aligned}$$

Definition 2. The set of *free variables* of a SPARQLTree expression φ is denoted $\text{Var}(\varphi)$ and is defined inductively as $\text{Var}(\texttt{PRED}(h,p,t)) = \{h,p,t\} \cap \mathcal{X}$, $\text{Var}(\texttt{AND}(\textit{body}_1,\textit{body}_2)) = \text{Var}(\textit{body}_1) \cup \text{Var}(\textit{body}_2)$, $\text{Var}(\texttt{UNION}(\textit{body}_1,\textit{body}_2))) = \texttt{Var}(\textit{body}_1) \cap \texttt{Var}(\textit{body}_2)$,and $\text{Var}(\texttt{FILTER}(\textit{body},fc)) = \text{Var}(\textit{body})$.

The SPARQL language definition is intricate, and we have chosen here to present a simplified version to preserve clarity. For example, the definition of `UNION` means some variables may be unbound. For that reason, we have chosen to include in the free variables only those that must be bound in a solution. Beyond the implicit existential quantification on the variables of a query, nested universal and existential quantifiers are encoded using the `FILTER` construct, and thus variables introduced in the `EXISTS` part of the filter constraint are removed when computing the solutions.

Given a formula $\text{PRED}(h,p,t)$ and a knowledge graph G we denote $s(\text{PRED}(h,p,t),G)$ the largest set of substitutions of domain $\text{Var}(\text{PRED}(h,p,t))$ such that for every $\sigma \in s(\text{PRED}(h,p,t),G)$ we have $\text{PRED}(h,p,t)\sigma \in G$.

Definition 3. (Solutions of a SPARQLTree body) The application of a SPARQLTree formula φ on a KG G yields a set of substitutions $\text{Sol}_G(\varphi,\Sigma_\emptyset)$ of domain $\text{Var}(\varphi)$ defined inductively on φ as follows:

$$\begin{aligned} \text{Sol}_G(\text{PRED}(h,p,t),\Sigma_D) &= \Sigma_D \bowtie s(\text{PRED}(h,p,t),G) \\ \text{Sol}_G(\texttt{AND}(\varphi_1,\varphi_2),\Sigma_D) &= \text{Sol}_G(\varphi_2,\text{Sol}_G(\varphi_1,\Sigma_D)) \\ \text{Sol}_G(\texttt{UNION}(\varphi_1,\varphi_2),\Sigma_D) &= \pi_{(\text{Var}(\varphi_1)\cap\text{Var}(\varphi_2))\cup D}(\text{Sol}_G(\varphi_1,\Sigma_D) \cup \text{Sol}_G(\varphi_2,\Sigma_D)) \\ \text{Sol}_G(\texttt{FILTER}(\varphi,fc),\Sigma_D) &= \pi_{D'}(\text{Sol}^c_G(fc,\text{Sol}_G(\varphi,\Sigma_D))) \\ &\qquad \text{with } D' \text{ domain of } \text{Sol}_G(\varphi,\Sigma_D) \end{aligned}$$

The computation of solutions with constraints $\text{Sol}_G(\varphi,\Sigma_D)$ is defined with:

$$\begin{aligned} \text{Sol}^c_G(\texttt{NOT}(\varphi),\Sigma_D) &= \Sigma_D \setminus \pi_D(\text{Sol}^c_G(\varphi,\Sigma_D)) \\ \text{Sol}^c_G(\texttt{EXISTS}(\varphi),\Sigma_D) &= \pi_D(\text{Sol}_G(\varphi,\Sigma_D)) \end{aligned}$$

A complete and more general computation of solutions is given in Polleres and Wallner (2013), but

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

Definition 3 is the basis of the algorithms provided *infra* aiming at explaining the lack of solutions of a query on a KG G.

In addition to the explicit rules extracted from regulations, we also need to integrate in our rule base potential default cases, expressed implicitly in legal texts. We also have to integrate conflict-solving rules to implement the defeasible aspect of legal reasoning.

3.2. Identifying implicit rules and conflict-solving rules

In addition to the explicit rules, it is necessary to make explicit, with the help of legal experts, rules that are either implicit or that allow to solve conflicts between other rules. Those conflicts can have several causes. For example one rule may be an exception to another, or a legal doctrine defining priority between two rules may have not been applied (lex superior, lex posterior. This can be illustrated with the article in Fig. 2. Once analyzed, this article states that "the processing of sensitive personal data is permitted ONLY where some conditions are met". Intuitively, this suggests that there is an implicit "default" rule stating that "the processing of sensitive personal data is forbidden" and the rule from the LED is an exception to this default rule. The formalization of "default" rule is based on the axiom isProhibition(LED10_default) and the FOL expression:

$Processing(action) \wedge InvolvesDataset(action, dataset) \wedge ContainsData(dataset, data) \wedge SensitivePersonalData(data) \rightarrow HasCompliance(LED10_default, situation)$

And the rule that resolves the conflict states that if both rules are complied with, only the exception has to be considered (represented by negating the compliance with the default rule here):

$HasCompliance(LED10_default, situation) \wedge HasCompliance(LED10, situation) \rightarrow \neg HasCompliance(LED10_default, situation)$

We therefore have a representation of legal rules based on the Semantic Web, thereby addressing our research question RQ2. The explicit rules form the core of our rule base and are direct formalizations of normative legal articles, while the implicit rules define default cases and the conflict-solving rules implement the defeasible aspect of legal reasoning. To justify the use of SPARQL, we have formally defined the elements of tree-based SPARQL syntax, SPARQLTree, noting that translations between FOL and SPARQL are possible via ASP.

4. Reasoning and explainability

The extraction of explicit rules and the identification of implicit, exceptions and priority solving rules generate a full rule base on which to reason. The reasoning itself is decomposed into two steps:

1. All the explicit and implicit rules are applied on the situation KG, and if the premises of a rule are satisfied, its conclusion and an indication that it was satisfied are added to the situation KG;
2. All the exception and priority rules are applied on the obtained KG to solve potential conflicts by removing the decisions of the rules that are superseded by other rules.

The reasoning aims not only at checking the compliance of a case with the regulation and at suggesting a decision to the end user, but it will also produce an explanation of this decision. This module of the

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

framework deals with RQ3.

At this point, the framework possesses a list of legal rules that apply to the input situation. The legal texts to be formalized within the framework are laws and regulations that specify what can and cannot be done in a given context. Numerous studies have examined the use of deontic logic to formally express this type of text (Jones and Sergot, 1992; Navarro and Rodríguez, 2014). Although some works have explored complex relationships between different deontic modalities (Moretti, 2009), we base our study on a standard "deontic square" comprising four modalities that can be linked by a negation relation: The Permission (P) and its opposition the Prohibition/Interdiction (I), the Obligation (O) and its opposition the Facultative (F). Each normative rule formalized in the framework can be classified into one of these four categories.

Thus, the list of satisfied rules generated by the reasoning module also provides the deontic modalities of those rules. In order to reach an overall conclusion regarding the situation entered by the user, it is necessary to understand how the various modalities interact, which will be detailed in the next section.

4.1. Evaluating the resulting deontic conclusion

Establishing the resulting deontic conclusion from a set of respected rules requires to analyze all the possible combinations of deontic types.

To do that, we built a deontic truth table 1, using the principles of the deontic square of oppositions (Moretti, 2009). It summarizes the relations between each combination of deontic types and the resulting global deontic conclusion, and its content reads as follows: Each column represents a possible combination of deontic types, where *1* denotes the presence of rules of this type in the list of respected rules, and *0* the absence of rules of this type in the list. The letters *F*, *O*, *I* and *P* stand respectively for *Facultative* (the right to not do), *Obligation*, *Interdiction* and *Permission* (the right to do). In the result column, an $\emptyset$ represents a situation where no single rule is respected, and $\times$ highlights all the cases in which the respected rules lead to an inconsistent decision in deontic terms.

Table 1: Truth table for the global deontic conclusion of a set of deontic rules

Case	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Facultative	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
Obligation	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
Interdiction	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
Permission	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Result	$\emptyset$	P	I	$\times$	O	O	$\times$	$\times$	F	P and F	I	$\times$	$\times$	$\times$	$\times$	$\times$

For example, Case 5 corresponds to a situation where some remaining decisions are Obligations, and others are Permissions (there is a 1 in the *O* and the *P* columns, 0 in the others). In this situation, the deontic notion of obligation prevails over the permission, resulting in an Obligation result. In Case 3, the remaining decisions are of the Permission and Interdiction types (there is a 1 in the *I* and the *P* columns, 0 in the others). In this situation, interdiction and permission are conflicting deontic conclusions, and the computed result is a contradiction, denoted with the $\times$ symbol.

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

It can be noted that one specific combination in this truth table that combines only Permission and Facultative rules results in a double deontic conclusion, stating that the action is neither obligated nor prohibited (line 9). Another point of note is that Table 1 distinguishes between Facultative and Permitted by making the former compatible with an interdiction and the latter compatible with an obligation.

The results from Table 1 can be organized more succinctly in a Karnaugh-like map as in Table 2. This representation allowed us to find a minimal simplified combination of deontic types to compute the conclusion of the rule-based reasoning module (*cf.* logical formulas (1), (2), (3), (4), (5) and (6)).

Table 2: Deontic Karnaugh table

		FO			
		00	01	11	10
IP	00	$\emptyset$	O	$\times$	F
	01	P	O	$\times$	P & F
	11	$\times$	$\times$	$\times$	$\times$
	10	I	$\times$	$\times$	I

$$P = P \cdot \overline{I} \cdot \overline{O} \tag{1}$$
$$F = F \cdot \overline{I} \cdot \overline{O} \tag{2}$$
$$O = O \cdot \overline{I} \cdot \overline{F} \tag{3}$$
$$I = I \cdot \overline{P} \cdot \overline{O} \tag{4}$$
$$\times = I \cdot P + F \cdot O + O \cdot I \tag{5}$$
$$\emptyset = \overline{P} \cdot \overline{I} \cdot \overline{O} \cdot \overline{F} \tag{6}$$

The formula (1) indicates that the resulting deontic conclusion is a permission if the list of respected rules contains permission rules and no interdiction and no obligation rules. Formulas (2), (3) and (4) are the analogous formulas for respectively facultatives, obligations and interdictions. The formula (5) indicates that there is conflict in three configurations: (i) There are Interdiction and Permission rules in the list; (ii) There are Facultative and Obligation rules in the list; (iii) There are rules classified as Obligation and Interdiction in the list. Finally, the last formula (formula (6)) clearly expresses the fact that no rule is being respected, which forms the undecided case that is addressed *infra*.

In conclusion, this deontic reasoning can thus generate 7 different output values that derive from 3 more general types of results. First, cases where at least one rule is satisfied by the input situation, with all the satisfied rules giving consistent decisions such that it is possible to draw a clear overall deontic conclusion (F, O, I, P, as well as P & F). Secondly, cases where at least 2 rules are respected but their conclusions are inconsistent, creating a contradiction ($\times$). And lastly, undecided cases in which no compliance rule is satisfied by the input ($\emptyset$). Each of these cases is handled in a different way, allowing to write an algorithm on how to handle the output of the reasoning mechanism.

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

4.2. General algorithm for explainability

The explanation given to the user will depend on the type of results from the reasoning phase, following the logic illustrated in Algorithm 1 that allows us to answer research question RQ3. When all the respected rules are coherent, the explanation is straightforward and consists of informing the user of the regulation parts that support the result. However, two problematic cases may arise. The first is when the rules produce contradictory decisions after the second conflict-solving step. This indicates that these rules are missing from the rule base. The result returned to the user highlights the contradiction, indicates the conflicting rules and asks him which rule should prevail. The answer is added as a new, temporary conflict-solving rule in the system, and is put forward for validation by an expert. The second problematic case arises when no compliance rule from the rule base has been respected. If no applicability rule is satisfied, the system indicates that the input situation is not in the scope of the verified regulations. The next section describes the cases in which some applicability rules are satisfied but no compliance rule.

Algorithm 1: Explanation of the rule-based reasoning results

Data: $(listApplicableRules, listRespectedRules)$; `/* Result from reasoning */`
Result: $(result, explanation)$; `/* The justified decision suggestion */`
if *listApplicableRules is empty* **then** `// There are no applicable rules`
  return "The situation is outside the scope of applicability of the verified regulations" ;
else if *listRespectedRules is empty* **then** `// None of the applicable rules are respected`
  $result \leftarrow undecided$;
  ask the user for potentially missing information to try to improve the result ;
else
  if *all rules in listRespectedRules are Permission* **then**
    return (Permission, listRespectedRules) ;
  else if *all rules in listRespectedRules are Facultative* **then**
    return (Facultative, listRespectedRules) ;
  else if *all rules in listRespectedRules are either Permission or Facultative* **then**
    return (Permission & Facultative, listRespectedRules) ;
  else if *all rules in listRespectedRules are either Obligation or Permission* **then**
    return (Obligation, listRespectedRules) ;
  else if *all rules in listRespectedRules are either Prohibition or Facultative* **then**
    return (Prohibition, listRespectedRules) ;
  else `// There are contradictory respected rules`
    return (Contradictory, listRespectedRules) ;
  end
end

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

5. Handling undecided cases using decision trees

In the case where no rule from the ruleset is respected, the system will ask the user for complementary information regarding the input situation. In order to determine which information to request from the user, the idea is to check what part of the rules that were "the closest to being complied with" were not respected. To do this, the following procedure is applied: (i) Consider only the rules "applicable" to the situation. (ii) For each of these rules, generate a binary decision tree where each node is one of the conditions of the rule, the left edge of a node corresponds to "the condition of the node is respected", and the right edge to "the condition of the node is not respected". (iii) Confront the input situation to each tree and keep a track of the non-respected conditions in them. (iv) sort the rules depending on how deep in their decision tree the verification went. (v) Use the non respected conditions in each rule to ask the user complementary information regarding the situation in input.

If the user is able to give supplementary information, the system processes the updated input from the beginning of the framework, hoping to obtain a *"decided"* decision. After this new processing,if the reasoning fails again to find respected rules or if the user cannot give supplementary information, the case is definitely classified as *"undecided"*.

The structure of the decision trees proposed in this section contributes to identifying relevant information to ask the end user, answering RQ4. In addition, the reasoning about undecided cases is an answer to RQ3 as it participates in providing better explanations to the end user.

In the following section, we will detail the principle behind the tree construction (step 2 in the procedure), and then we will explore two approaches for confronting the input situation to the trees (step 3 of the procedure).

5.1. Architecture of the decision trees

The idea behind the constructed trees is to use their traversal to check the conditions of each applicable rule individually and establish which specific conditions were not fulfilled during the reasoning phase.

Each tree is a binary tree and implements an applicable rule. Each node corresponds to one of the rule conditions, with the root node representing the first condition. Left edges correspond to a positive evaluation of a condition, while the right edges correspond to its negative evaluation. Each leaf on the tree may represent one of two types of results: (i) A positive output would indicate that all the rule conditions have been met for it to be respected. This type of leaf will never be reached during tree traversal because the construction of these trees is precisely motivated by the fact that no rule is being respected. (ii) A negative output contains the condition or the set of conditions that have not been met, explaining why the associated rule, although applicable, was assessed as not being respected during the reasoning process.

Analysis of failure. Def. 3 provides a total order of evaluation of the parts of a query. let us analyze the first failure according to the different cases with the notations of that definition (unrolling the cases of filter constraints):

`PRED` : Assuming $\Sigma_D \neq \emptyset$ (first failure), the result is an empty set only if no instance of the predicate in G is joinable with a substitution in Σ_D. In that case the explanation is the predicate;

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

`AND` **and** `UNION` : A failure cannot occur at this level, it has to occur during the computation of either subformulas of `AND` or both subformulas of `UNION`;

`FILTER EXISTS` : This case behaves as an `AND` node with an added projection that cannot make the set of solutions empty;

`FILTER NOT EXISTS` : This case results in an empty set of solutions if $\Sigma_D \subseteq \pi_D(\text{Sol}_G(\varphi, \Sigma_D))$, *i.e.* if **all** substitutions in Σ_D can be extended to satisfy φ.

The last case often occurs in the translation of universal subformulas, and in that case φ is itself of the form $\texttt{FILTER}(\varphi_1, \texttt{NOT}(\texttt{EXISTS}(\varphi_2)))$. To provide a better explanation in that case, we introduce a new node `UNIVERSAL` of arity 2. Noting that a first-order logic formula $\varphi_1 \wedge (\forall x, \varphi_2 \Rightarrow \varphi_3)$ is translated into $\texttt{FILTER}(\varphi_1, \texttt{NOT}(\texttt{EXISTS}(\texttt{FILTER}(\varphi_2, \texttt{NOT}(\texttt{EXISTS}(\varphi_3))))))$, all such patterns are rewritten from the bottom-up into $\texttt{AND}(\varphi_1, \texttt{UNIVERSAL}(\varphi_2, \varphi_3))$. By unrolling Def. 3, we find that:

$$
\begin{aligned}
&\text{Sol}_G(\texttt{UNIVERSAL}(\varphi_1, \varphi_2), \Sigma_D) \\
&\quad = \text{Sol}_G(\texttt{FILTER}(\top, \texttt{NOT}(\texttt{EXISTS}(\texttt{FILTER}(\varphi_1, \texttt{NOT}(\texttt{EXISTS}(\varphi_2))))))), \Sigma_D) \\
&\quad = \Sigma_D \setminus \pi_D(\text{Sol}_G(\texttt{FILTER}(\varphi_1, \texttt{NOT}(\texttt{EXISTS}(\varphi_2))), \Sigma_D)) \\
&\quad = \Sigma_D \setminus \pi_D(\text{Sol}_G(\varphi_1, \Sigma_D) \setminus \pi_{D'}(\text{Sol}_G(\varphi_2, \text{Sol}_G(\varphi_1, \Sigma_D)))) \\
&\quad = \Sigma_D \setminus (\pi_D(\text{Sol}_G(\varphi_1, \Sigma_D)) \setminus \pi_D(\text{Sol}_G(\varphi_2, \text{Sol}_G(\varphi_1, \Sigma_D))))
\end{aligned}
$$

Consider a substitution $\sigma \in \Sigma_D$. It does not occur in the solution if it is present in $\pi_D(\text{Sol}_G(\varphi_1, \Sigma_D))$—meaning extra variables can be instantiated to satisfy the condition of the filter—but not in $\pi_D(\text{Sol}_G(\varphi_2, \text{Sol}_G(\varphi_1, \Sigma_D)))$—meaning the conclusion of the universal cannot be satisfied with these additional instances. We note that failing to satisfy the condition φ_1 of the universal is not a cause of failure. *I.e.*, the explanation of the failure of a `UNIVERSAL` node is that of the failure of φ_2.

Let us now consider the origin of these rules. We assume that, as in our example, universal quantification is employed to constrain (using φ_2) the items described by φ_1. These constraints are naturally a conjunction of properties that have to be satisfied. Instead of exploring *why* a given property failed, we believe it is sufficient to handle the property as a whole. The same principle holds for the `FILTER NOT EXISTS` construct. We propose to handle this structure of explanation by constructing an *explanation forest* to structure the search for the explanation of failure by adding sequence points in the sequential evaluation of the query.

Definition 4. (Explanation tree, explanation forest) An *explanation tree* is either:

- A `NORMAL_block` with a list of predicates;
- A `UNION_block` with a list of explanation trees whose root is not a `UNION_block`;
- A `FILTER_NOT_EXISTS_block` with two lists, one of explanation trees called its condition, and one of SPARQLTree formulas called its *consequence*;
- A `UNIVERSAL_block` with a SPARQLTree formula called its *condition*, and a list of SPARQLTree formulas called its *consequence*;

An *explanation forest* is a list of *explanation trees*.

A SPARQLTree query body is translated into an explanation forest. Its construction from a SPARQLTree formula is given in the appendice**/is routine(if not in the appendice)**.

Definition 5. (Decision tree) A *decision tree* is either a `OK` node, a `KO` node, or a `F`(ormula) node labeled with a SPARQLTree formula with two sons labeled one with `YES` and the other with a text providing an

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

explanation.

Each explanation forest is translated into one decision tree with exactly one `OK` and one `KO` node. This transformation algorithm performs operations on nodes in trees, and we believe it is not particularly illuminating. We have presented a full translation example in Fig. 3, and described the translation patterns for each kind of explanation tree nodes in Fig. 6.

To illustrate what such a tree looks like, we will use simple arbitrary letters a, b, c, etc. to represent conditions or groups of conditions in the nodes. Starting from the root, recursively, the construction of the tree follows these principles:

- When the condition a in the current node and the next condition b are connected by a logical $\wedge$ operator (structure $a \wedge b$), the node containing b is connected to the left branch leaving from the node a. In this case, it is because, if condition a is not verified, then $a \wedge b$ as a whole is not verified. On the other hand, if a is verified, in order to verify $a \wedge b$, it is still necessary to test whether b is verified.
- When the condition a in the current node and the next condition b are connected by a logical $\vee$ operator (structure $a \vee b$), the node containing b is connected to the right branch leaving from the node a. In this case, it is because if condition a is already verified, then $a \vee b$ as a whole is also verified. On the other hand, if a is not verified, in order to verify $a \vee b$, it is still necessary to test whether b is verified.
- The branch leaving from a that is not yet connected is then connected either to a leaf or, if there are still other conditions connected with $\vee$ operators later in the rule, to the first node of the set of conditions that follows the block currently being processed.

Let's consider an arbitrary example of a simplified SPARQL query representing a rule with conditions expressed as simple predicates. Since the head is irrelevant for the construction of the tree, we will only consider the body of the query. In First-Order Logic, the rule would be written as $a \wedge b \wedge (c \vee (d \wedge \neg e) \vee (\forall x.(f(x) \wedge g(x)) \implies (h(x) \wedge i(x)))) \wedge j$, which in SPARQL would be expressed as shown in listing 2.

Listing 2: Body of a simplified SPARQL rule to illustrate the construction of the tree

```
a .
b .
{c .}
UNION
{
    d .
    FILTER NOT EXISTS{e .}
}
UNION
{
    FILTER NOT EXISTS{
        f .
        g .
        FILTER NOT EXISTS{
            h .
            i .
        }
    }
```

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

```
}
j .
```

The query in listing 2 contains all the structures that can be found in a legal article: conjunctions and disjunctions of conditions, negations, universal quantification, some of them nested in each other. In the corresponding SPARQL query, conjunction and disjunction are written using respectively the dot and the UNION keyword. Universal quantification is encoded using the FILTER NOT EXIST construction. For example, the second UNION of the query represents a condition *"All the elements that verify f and g, are such that they also verify h and i"* that can be found in a legal rule "*All the data that are personal and sensitive, are such that they also are public and disclosed by the person concerned*". In SPARQL, this universal quantification is expressed with two negated existential operators, literally saying that *"there is no f and g without also having h and i"* or, in the example, it can be rephrased as ("*There is no data that is personal and sensitive, that is not also public and disclosed by the person concerned*"). A side-effect of rephrasing the universal quantification is to allow a more precise identification of the rule parts that could be violated. The tree constructed from this rule and following these principles is shown in Fig. 3.

If we apply the same principle to our running example of article 10 from the LED, we obtain the tree in figure 4.

5.2. Constructing the trees from the SPARQL queries

Now that we have established the principles behind the architecture of the decision trees, we need a procedure that allows us to transform SPARQL rules into this tree form.

First, the construction of the decision tree for each applicable rule requires parsing the related SPARQL rules about compliance. This parsing is based on breaking down the SPARQL query into blocks of different types that can either be sequential or nested within each other. The type of blocks may be `NORMAL_Block`, `NEGATION_Block`, `UNIVERSAL_Block` and `UNION_Block`, each of which is defined by different attributes:

- The `NORMAL_Block` is the most basic element to be parsed. It is defined by a list of strings, each of these strings being one of the triples of the query.
- The `NEGATION_Block` is used when a part of the query is inside a simple *"FILTER NOT EXISTS"* structure. It is defined by the list of the blocks inside the *"FILTER NOT EXISTS"*.
- The `UNIVERSAL_Block` allows to parse universal quantification in queries, that are expressed with two nested *"FILTER NOT EXISTS"* in SPARQL. This block is defined by two lists of blocks. The first list corresponds to the part inside the first *"FILTER NOT EXISTS"* of the structure and the second list to the elements inside the second *"FILTER NOT EXISTS"* structure.
- The `UNION_Block` represents all the alternatives of a *"UNION"* structure in SPARQL. The datatype used to characterize it is a list of lists of Blocks. Indeed, a *"UNION"* structure is composed of several alternatives, each of which is a sequence of blocks.

Finally, the query itself is parsed as a list of blocks. To illustrate the parsing, let us consider the SPARQL structure from listing 2. Parsing this example results in the following rule decomposition:

The query itself is composed of a succession of 3 main blocks, a `NORMAL_Block` followed by a `UNION_Block` and finally another `NORMAL_Block`:

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

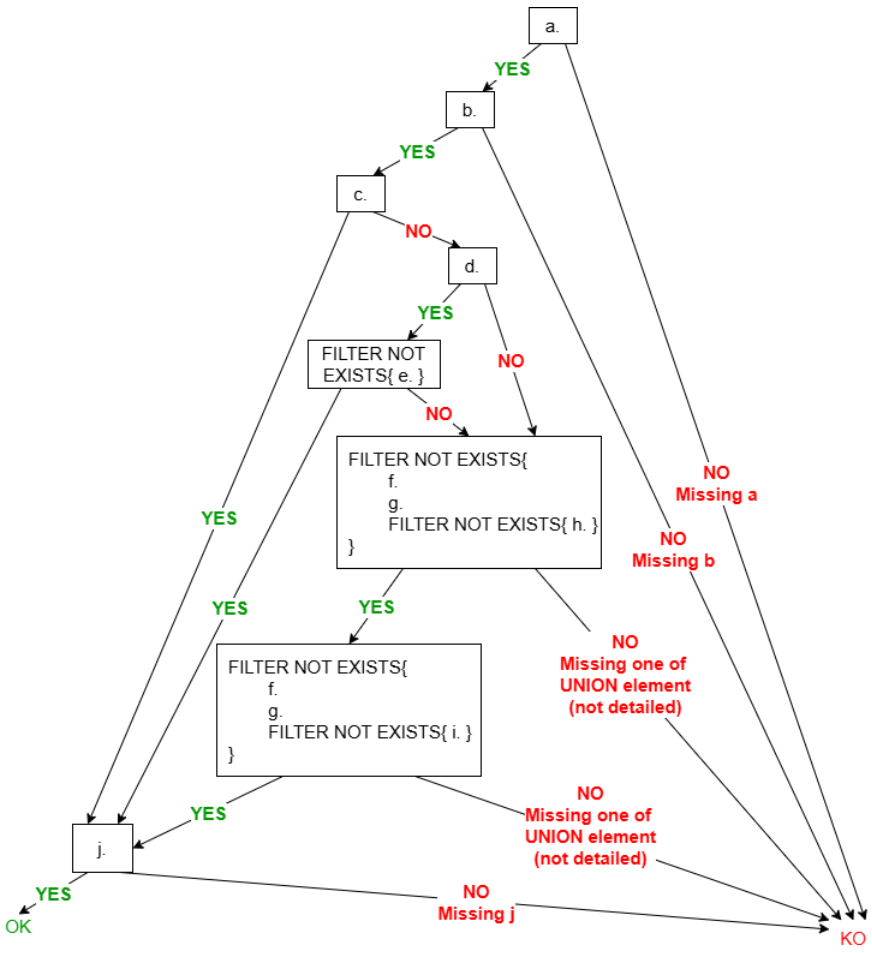


Fig. 3: Tree to determine the unfulfilled condition of the arbitrary rule from listing 2

- The first `NORMAL_Block` contains the two conditions a and b.
- The `UNION_Block` is composed of three alternatives:
  - The first alternative is a simple `NORMAL_Block` with the condition c.
  - The second alternative contains a `NORMAL_Block` followed by a `NEGATION_Block`:
    - The `NORMAL_Block` contains only the condition d.
    - The `NEGATION_Block` contains a single `NORMAL_Block` whose content is e.
  - The third alternative contains a `UNIVERSAL_Block`:
    - The first part of the `UNIVERSAL_Block` is composed of a single `NORMAL_Block` that contains f and g.
    - The second part of the `UNIVERSAL_Block` contains also a single `NORMAL_Block`, composed of the conditions h and i.

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

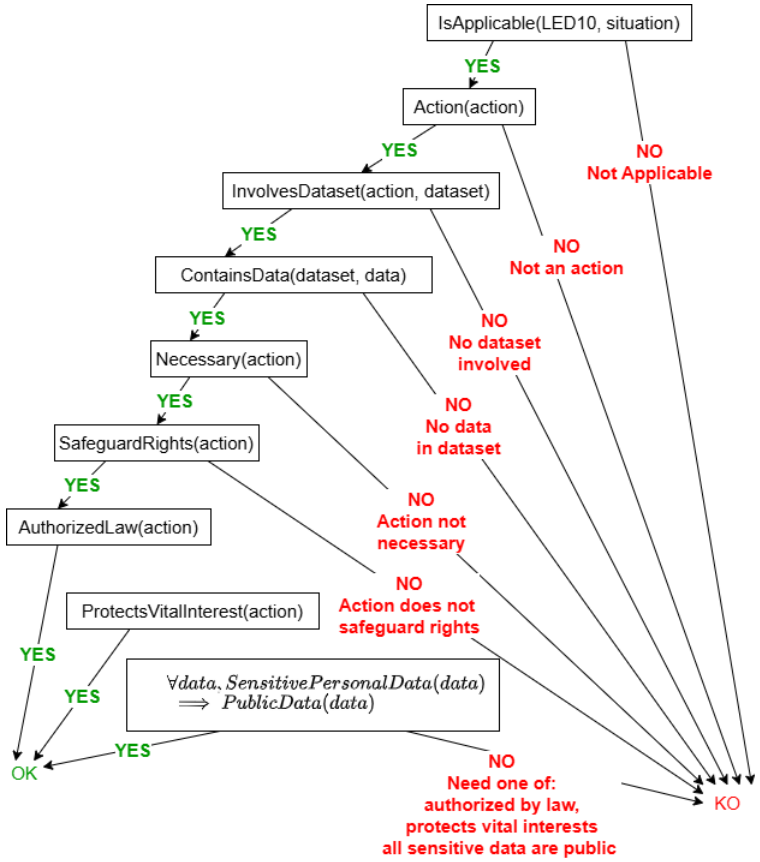


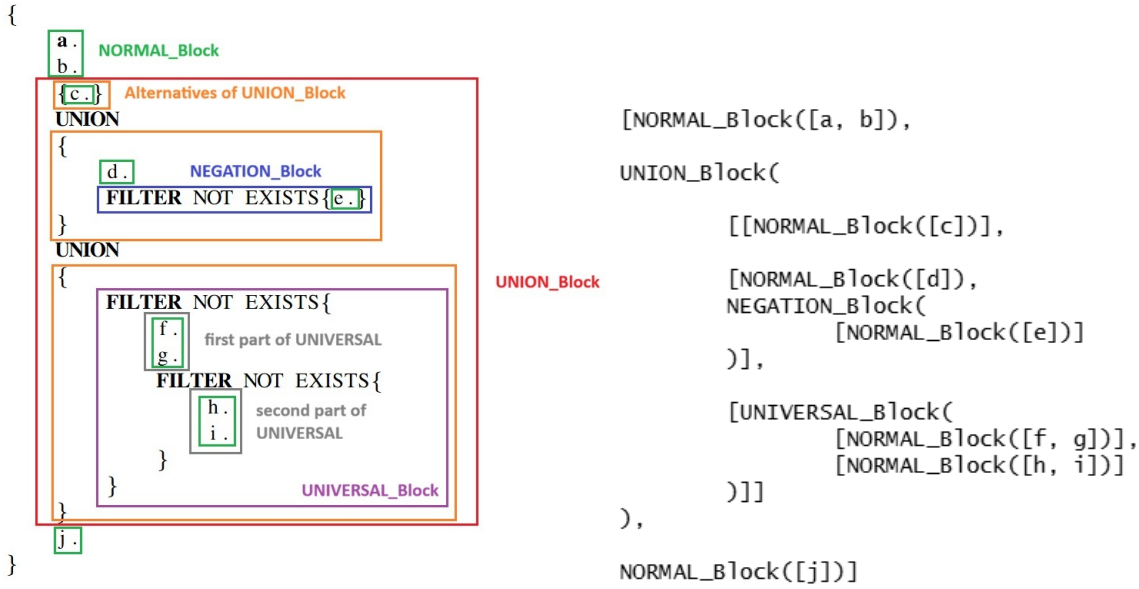
Fig. 4: Tree to determine the unfulfilled condition of the rule that checks for compliance with article 10 of the LED

- The last `NORMAL_Block` contains the last condition j.

Fig. 5a illustrates the different blocks that compose this example. Block identification enables the construction of a decision tree based on specific patterns that keep a one-to-one correspondence between the tree nodes and the rule structure and conditions.

- The pattern to build a node from a `NORMAL_Block` is the simplest. It consists of chaining the conditions in the block through left branches (the branches for a positive checking of the condition). An example for the block `NORMAL_Block([a, b])` is given in Fig. 6a.
- Building a node from a `NEGATION_Block` only consists of prefixing the content of all the nodes constructed from the blocks inside the negation, as illustrated in Fig. 6b for the example `NEGATION_Block([Block_A, Block_B])`.
- Building a node from a `UNIVERSAL_Block` is also straightforward, and consists of chaining the subtrees created from the blocks in the second part of the universal structure. The result of such construc-

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies



(a) Blocks that compose the rule in the studied example

(b) Textual translation of the parsing

Fig. 5: Parsing of the blocks that compose the query used as example

tion from the example `UNIVERSAL_Block(first, [Block_A, Block_B])` is given in Fig. 6c.

- The pattern to build a node from a `UNION_Block` is essentially the mirrored principle compared to a `NORMAL_Block`. Each subtree created from the alternatives of the `UNION` are chained through the right branches (the branches for a negative checking of the condition). For example, the tree created from example `UNION_Block([[Block_A], [Block_B], [Block_C]])` is given in Fig. 6d.

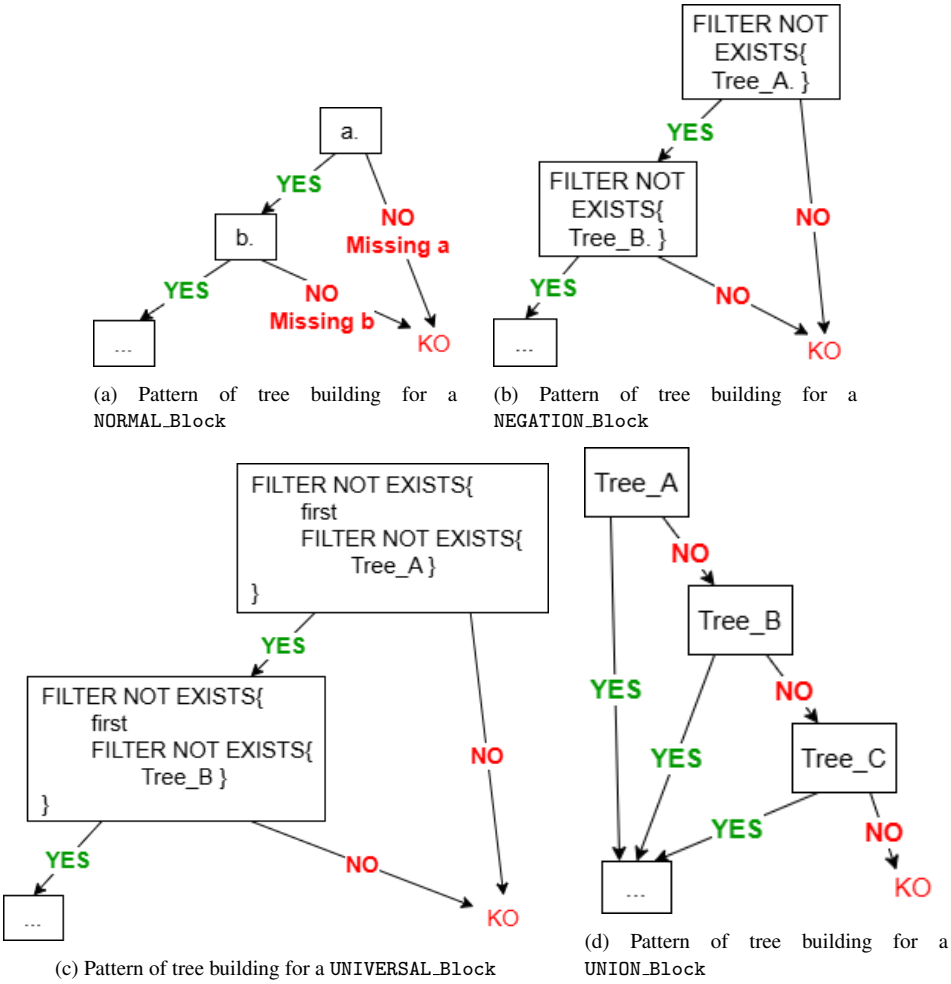
Next, when going through the trees, it is necessary to establish how each node will be evaluated in order to take the correct branch at each step. The evaluation of a node is actually carried out by executing a SPARQL query designed to check specifically the condition inside the node. These queries are not INSERT queries like the ones used in the first module of the framework, but ASK queries. ASK is a SPARQL keyword that allows to test whether or not a query pattern has a solution, without returning information about the possible solutions. The binary response of these queries are then directly used as a way to determine which tree branch to take from each node. However, building the query pattern is challenging, as it requires to keep a track of the conditions that have already been checked as satisfied.

Fig. 7 gives an example of the queries constructed in each node of the tree in Fig. 3.

We can note that three possible SPARQL queries can be executed in the last node. This is because prior to this node was a `UNION` with three alternatives. These alternatives are checked in order until one of them is verified. If one of the alternatives is verified, verification of others is skipped, and the conditions that are part of the verified alternative are registered in order to construct the SPARQL query for the following nodes.

When an *undecided* conclusion is reached, a tree is constructed from each applicable rule in our

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies



(a) Pattern of tree building for a `NORMAL_Block`

(b) Pattern of tree building for a `NEGATION_Block`

(c) Pattern of tree building for a `UNIVERSAL_Block`

(d) Pattern of tree building for a `UNION_Block`

Fig. 6: Patterns of tree building depending on the type of block parsed

system following the patterns presented. An algorithm then explores this forest to identify the conditions that are not actually met by the input situation.

5.3. Exploring the forest to identify unfulfilled conditions

Several approaches are possible for traversing the trees, in order to identify both the rules that were the closest to being respected and the unfulfilled conditions within these rules. We will explore two of them: (i) A first, naive approach, that looks only for the first set of decisive conditions that are not met in each applicable rule. (ii) A second, more optimized approach, which consists of searching for the largest subset of conditions that are satisfied by each applicable rule.

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

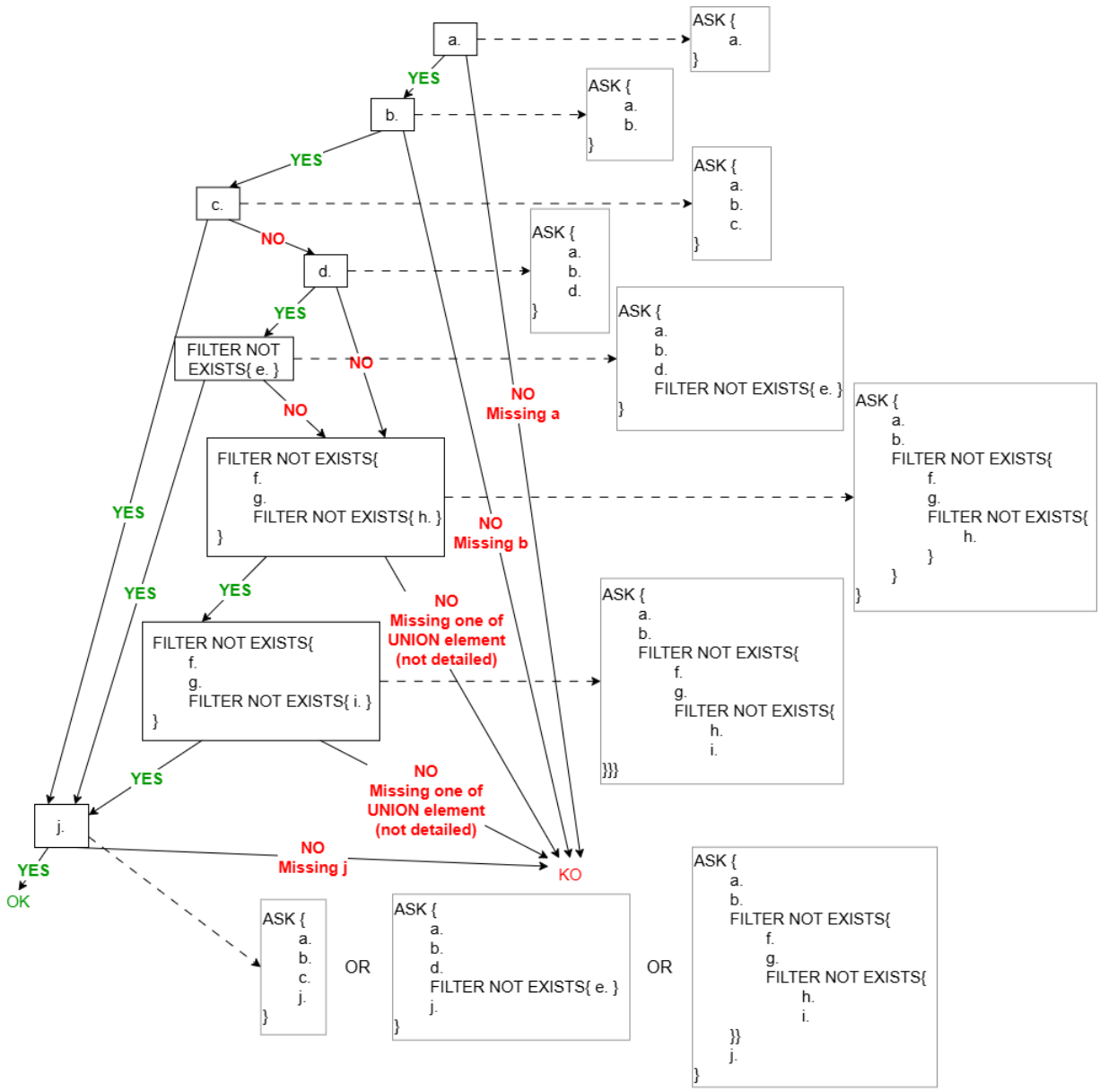


Fig. 7: SPARQL queries executed to check each node of the tree

5.3.1. A naive approach: Seeking the first unfulfilled decisive conditions

On a concrete example, Fig. 4 illustrates the tree that would be created from the compliance rule obtained from article 10 of the LED (cf. Fig. 2). For clarity and intelligibility, the nodes of this tree contain the conditions written in a more compact syntax, equivalent to what is written in SPARQL. The first

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

approach aims at identifying the first unfulfilled decisive conditions in each applicable rule to the input situation. It can be noted that when the law articles are formalized, the conditions specified in the article, once translated, are listed in the same order in the SPARQL query than in natural language. This order tends to follow an order of increasing precision, where the further one goes into a article of law, the more requirements there are to verify, and the more detailed those requirements become. It is therefore relevant to consider that the further down the tree we go, the closer we are to fully complying with the rule.

We call *decisive conditions* a subset of the conditions of the rules such that failure to meet all the conditions in this subset is necessary and sufficient for the rule to be violated. All leaves of the tree that are not a positive outcome (leaves that are not labeled *"OK"* in our examples) contain one such subset. In other words, finding the first unfulfilled decisive conditions simply consists of finding the leaf reached when confronting a situation with the tree.

Given a tree created from a rule and a system input situation, the general principle is to go down the tree starting from the root node. For each node, take the branch that corresponds to the result of the binary check of compliance with the condition specified in that node. We stop when a leaf is reached, which gives us the first set of unfulfilled conditions of the rules.

Repeating the process for each tree created from the applicable rules gives us, for each of these rules, their set of unmet conditions as well as the depth of the leaf that was reached. From the depth reached and the maximum depth of the tree, we can calculate the relative depth reached, which can then be used as a metric to sort the rules in descending order of its value.

An algorithm that describes the implementation of this naive approach is provided in Appendix A.

However, this naive approach has two major flaws:

1. It stops the validation of the conditions at the first unfulfilled decisive identified conditions, and doesn't check any further. By doing this, there is no guarantee that the information that the user is asked to provide will be sufficient to validate the rules. For instance, in our example, the procedure stopped after finding a missing information regarding the necessity of the action. But the end of the rule, with a disjunction of conditions, is not met either. With the naive approach, this lack will only be identified after the user has provided the required information about *necessity* and the system has been run again.
2. Using the relative depth reached is not an optimal metric either for ranking rules in order of their closeness to being satisfied. For example, a rule can start with a part establishing a certain context, *e.g.* that the user must be a police officer. That context is irrelevant in the evaluation of the rest of the rule, and a failure can just be the result of an unintended omission. However, with the naive approach, this rule would be put aside in terms of *"closeness to being fulfilled"*. To formally express the issue, let us consider a situation that must be compared against two rules, both of which have the same number of conditions to check. The trees of these rules would then have the same maximum depth. For the first rule, let us assume that the situation meets all conditions except one, and this condition is very high up in the tree, among the first conditions checked. For the second rule, let us suppose that the situation meets almost all the conditions, except 3, but all these conditions are at the end of the tree, and are the very last conditions checked. According to our metric, the rule with the higher relative depth reached would be the second one, and yet, the first rule, with only 1 unfulfilled condition, should be considered *the closest of being respected.*

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

Although the naive approach proves to be sufficient in simple cases, the two flaws mentioned above make it ineffective in more complex cases, with rules involving numerous conditions to be checked to establish a proper context for the application of the rule. This observation leads to the establishment of a more optimized strategy presented in the next section.

5.3.2. An optimized approach: Seeking all the unfulfilled conditions

This second approach aims at identifying not just the first, but all the unfulfilled decisive conditions in each applicable rule to the input situation. To do this, we need to check all the conditions of the rule, and note which ones are unfulfilled.

We achieve this by applying the following procedure to each rule:

1. Construct the tree associated with the rule.
2. Find and take note of the first unfulfilled conditions by applying the naive approach.
3. Create a new version of the rule where the unfulfilled conditions have been removed. Determining which part of the rule to remove is however not trivial. For example, if an unfulfilled condition is that a user is a member of a LEA, we should remove in the simplified version the dependent conditions that the LEA has jurisdiction over the place where the situation occurs, without removing the other conditions on that place. This step thus requires a dependency analysis among the conditions of the rule. The algorithm that carry out this analysis is described in details below.
4. Create a new tree from the updated version of the rule.
5. Check again for the first unfulfilled conditions.

This procedure is repeated until the leaf reached in the tree is a leaf that signifies the rule is respected (a leaf labeled *"OK"* in our examples).

By doing this, the initial rule gets split into two. On the one hand, a rule that is respected, composed of a subset of the conditions of the initial rule. We call it the *"Biggest respected sub-rule"*. On the other hand, there is the set of all the unfulfilled conditions of the rule, that can then be requested from the user all at once.

However, one aspect of this procedure requires special treatment. Indeed, as mentioned in the third point of the procedure, when creating the new version of the rule where the identified unfulfilled conditions are removed, a problem of dependency between variables can occur. The conditions expressed in SPARQL are in the form of RDF triples that link a subject with an object through a specific property, with some of the elements being part of an ontology, and the others being variables. If, during the checking of the individual conditions of a rule, an unfulfilled condition is found and that this condition introduces a new variable in the SPARQL query, getting rid of the condition also suppresses the introduction of the new variable, and further conditions that depend on this variable need to be removed from the rule as well before generating the tree. The difficulty lies in determining whether variables present in unfulfilled conditions are introduced into these conditions only.

To do so, we generalize our sequentiality assumption into a sequence-parallel assumption that the ordering on conditions is partial, but that along each path from the root new variables are introduced in relation with already determined variables. We can then build a dependency graph, where nodes are the variables of the rule, and edges represent links that exist between these variables in the conditions of the rule.

Generally, let us separate the conditions of a rule in three groups. (i) The conditions already validated,

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

(ii) the conditions that have just been evaluated as violated, and (iii) the conditions that have not been verified yet. When creating the updated version of the rule, all the conditions in the second group will be removed. Then we need to determine whether conditions in the third group should be removed too. To do this, we consider each condition in the third group in turn. For each condition, we note the variables that appear in it. Using our dependency graph, we can determine whether all the variables in the current condition depend on variables that have been removed. If that is the case, the current unverified condition must be removed too. Otherwise, this means that there exists a link between the variables in this condition and the variables in validated conditions. In this case, we can keep it.

6. Conclusions

This paper presented reasoning and justification methods for a decision support framework. We notably focused on answering four research questions regarding the use of symbolic AI and semantic web for formalizing regulations, and the generation of justified suggestions, requiring, in cases where the reasoning fails to reach a conclusion, to identify the most relevant information to ask the user. After describing the architecture of the framework and its components, we presented our approach formalizing legal rules. We then described the logic and algorithms behind the generation of a justified decision in our framework. Finally, to handle cases where not a single rule was satisfied, an innovative approach was presented to identify unfulfilled conditions in legal rules using decision tree structures. The use of the proposed framework has been illustrated in a use case of data sharing among European Law Enforcement Agencies (LEAs). Several issues are still to be addressed in future works. First, the number of formal rules used to test our framework shall be extended, in order to see how it scales up to bigger use cases. This could be done by extracting rules from new directives that have come into force in 2023 regarding the data processing by LEAs[10] and the gathering of electronic evidence in criminal proceedings[11]. We would also like to automate the extraction process of new formal rules, using state-of-the-art Natural Language Processing methods (Fawei, 2024; Ferraro et al., 2020; Recski et al., 2021). While the formal language used in this study is SPARQL, other formal standards like LegalRuleML (Palmirani et al., 2011) could be tested to compare the results. Looking further ahead, we are also aware that the tests conducted on our framework are currently limited to proof-of-concepts, and we would like to establish a full benchmark to conduct large-scale experiments using real-world data. Finally, we plan to eventually expand our framework with a case-based reasoning module. This would complement rule-based reasoning by leveraging learning derived from past decision traces.

Acknowledgments

This work is partially funded by the H2020 project STARLIGHT (“Sustainable Autonomy and Resilience for LEAs using AI against High priority Threats”) that received funding from the European

[10]Directive - 2023/977 - EN - EUR-Lex - European Union: https://eur-lex.europa.eu/legal-content/EN/HIS/?uri=oj:JOL_2023_134_R_0001

[11]Directive - 2023/1544 - EN - EUR-Lex - European Union: https://eur-lex.europa.eu/legal-content/EN/NIM/?uri=CELEX:32023L1544

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

Union's Horizon 2020 research and innovation program under grant agreement No 101021797. We would also like to thank Ronan PONS, PhD student in Law, who assisted this work by providing his insight as a legal expert.

References

Bouché-Pillon, J., 2025. A Decision Support System for Digital Law : An approach based on Ontologies, Deontic logic and Access Control. Theses, Université de Toulouse.

Bouche-Pillon, J., Aussenac-Gilles, N., Chevalier, Y., Zarate, P., 2024. Decision Support in Law: From Formalizing Rules to Reasoning with Justification. In *Frontiers in Artificial Intelligence and Applications*, Jaromir Savelka and Jakub Harasta and Tereza Novotna and Jakub Misek, IOS Press, Brno, Czech Republic.

Bouché-Pillon, J., Aussenac-Gilles, N., Chevallier, Y., Zaraté, P., 2024. An ontology for legal reasoning on data sharing and processing between law enforcement agencies. In *3rd international workshop KM4LAW – Knowledge Management and Process Mining for Law (2024)*, IAOA, p. TBP.

Fawei, B., 2024. Nlp-based rule learning from legal text for question answering. *Asian Journal of Research in Computer Science* 17, 7, 31–40.

Ferraro, G., Lam, H.P., Tosatto, S.C., Olivieri, F., Islam, M.B., van Beest, N., Governatori, G., 2020. Automatic extraction of legal norms: Evaluation of natural language processing tools. In *New Frontiers in Artificial Intelligence: JSAI-isAI International Workshops, JURISIN, AI-Biz, LENLS, Kansei-AI, Yokohama, Japan, November 10–12, 2019, Revised Selected Papers 10*, Springer, pp. 64–81.

Gandon, F., Governatori, G., Villata, S., 2017. Normative requirements as linked data. In *JURIX 2017-The 30th international conference on Legal Knowledge and Information Systems*, pp. 1–10.

Gordon, T.F., 2008. Constructing legal arguments with rules in the legal knowledge interchange format (lkif). In Casanovas, P., Sartor, G., Casellas, N. and Rubino, R. (eds), *Computable Models of the Law*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 162–184.

Jones, A.J., Sergot, M., 1992. Deontic logic in the representation of law: Towards a methodology. *Artificial Intelligence and Law* 1, 1, 45–64.

Lam, H.P., Hashmi, M., 2018. Enabling reasoning with legalruleml. *Theory and Practice of Logic Programming* 19, 1–26.

Lazarska, M., Siedlecka-Lamch, O., 2019. Comparative study of relational and graph databases. In *2019 IEEE 15th International Scientific Conference on Informatics*, pp. 000363–000370.

Lee, J., Palla, R., 2009. System f2lp–computing answer sets of first-order formulas. In *International Conference on Logic Programming and Nonmonotonic Reasoning*, Springer, pp. 515–521.

Marakas, G., 2003. *Decision Support Systems in the 21st Century*. Prentice Hall.

Medhi, S., Baruah, H.K., 2017. Relational database and graph database: A comparative analysis. *Journal of process management and new technologies* 5, 2.

Moretti, A., 2009. The geometry of standard deontic logic. *Logica Universalis* 3, 1, 19–57.

Navarro, P.E., Rodríguez, J.L., 2014. *Deontic logic and legal systems*. Cambridge university press.

Palmirani, M., Governatori, G., Rotolo, A., Tabet, S., Boley, H., Paschke, A., 2011. Legalruleml: Xml-based rules and norms. In Olken, F., Palmirani, M. and Sottara, D. (eds), *Rule-Based Modeling and Computing on the Semantic Web*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 298–312.

Pérez, J., Arenas, M., Gutierrez, C., 2009. Semantics and complexity of sparql. *ACM Transactions on Database Systems (TODS)* 34, 3, 1–45.

Polleres, A., Wallner, J.P., 2013. On the relation between sparql1. 1 and answer set programming. *Journal of Applied Non-Classical Logics* 23, 1-2, 159–212.

Recski, G., Lellmann, B., Kovacs, A., Hanbury, A., 2021. Explainable rule extraction via semantic graphs. In *ASAIL/LegalAIIA@ ICAIL*, pp. 24–35.

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

Appendix A

Algorithm 2: Tree traversal in the naive approach

Data: $query, tree, endpoint$; /* Body of the SPARQL query, root of associated tree, endpoint to execute SPARQL queries */
Result: $(node, unrespected)$; /* Node reached by traversing the tree, unrespected conditions */

$curr_query \leftarrow "ASK\{"$ // ASK query currently verified
$node \leftarrow tree$ // node initialized with the root
$unrespected \leftarrow []$
$temp \leftarrow ""$ // temporary part of ASK query
while *node is not a leaf* **do**
  if *node is the beginning of union sub-block* **then** // content of UNION temporary
    $temp \leftarrow ""$
    $temp_query \leftarrow curr_query$
    while *node not the end of UNION sub-block* **do**
      $temp \leftarrow temp + node.content$ // adding condition in the node to temp
      $temp_query \leftarrow curr_query + temp$ // adding temp to the query
      $result = ask_query_execution(temp_query, endpoint)$ // boolean result
      $(node, unrespected) \leftarrow next_node(result, node, unrespected)$ // see Alg. 3
    end
    // End of UNION sub-block
    $temp \leftarrow temp + node.content$
    $temp_query \leftarrow curr_query + temp$
    $result \leftarrow ask_query_execution(temp_query, endpoint)$
    $(node, unrespected) \leftarrow next_node(result, node, unrespected)$
    if *node is not the beginning of a new UNION sub-block* **then** // We finished traversing the whole UNION Block
      $curr_query \leftarrow temp_query$ // the temp query becomes definitive
  else // Not dealing with UNION
    $curr_query \leftarrow curr_query + node.content$
    $result \leftarrow ask_query_execution(curr_query, endpoint)$
    $(node, unrespected) \leftarrow next_node(result, node, unrespected)$
  end
end
return $(node, unrespected)$

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies

Algorithm 3: next_node: Function for advancing in the tree depending on the result of the ASK query

Data: $result, node, unrespected$; `/* boolean result of ASK query, current node, list of unrespected conditions */`
Result: $(node, unrespected)$; `/* New node, updated list unrespected */`
if *result* **then** `// ASK query returned true: condition verified`
  $node \leftarrow node.left$ `// condition OK: left branch`
else
  add node content to unrespected
  $node \leftarrow node.right$ `// condition NOK : right branch`
end
return $(node, unrespected)$

© 2026 International Transactions in Operational Research © 2026 International Federation of Operational Research Societies